

Compte rendu :Système de recherche distribué accéléré

GPU

Contexte et objectif

Ce TP a pour but de construire un **backend d'API de recherche vectorielle** capable d'exploiter la puissance de calcul parallèle d'un GPU (Tesla T4 sur Google Colab) pour effectuer des recherches par similarité cosinus sur un corpus de 10 000 documents, tout en restant fonctionnel en mode dégradé sur CPU si aucun GPU n'est disponible. L'API est ensuite exposée publiquement via un tunnel ngrok pour être consommée par une application frontend externe (React), ce qui en fait un TP orienté **système distribué** plus qu'un simple exercice de benchmark : il combine calcul GPU, exposition réseau et architecture client-serveur.

Le notebook est structuré en 5 cellules de code, chacune précédée d'une cellule markdown explicative très pédagogique, avec des messages d'erreur guidés et une gestion des cas d'échec pensée pour des étudiants qui n'auraient pas encore activé le GPU.

1. Installation des dépendances

```
!pip install flask flask-cors pyngrok cupy-cuda12x numpy
```

Quatre briques distinctes sont installées :

- **Flask** : serveur web léger pour exposer l'API REST.
- **flask-cors** : nécessaire car le frontend (React, servi localement) et le backend (Colab, exposé via ngrok) ne partagent pas la même origine — sans CORS, le navigateur bloquerait les requêtes.
- **pyngrok** : crée un tunnel HTTPS public vers le port local 5000 de Colab, contournant le fait que Colab n'expose pas nativement de port réseau accessible depuis l'extérieur.
- **cupy-cuda12x** : équivalent quasi drop-in de NumPy, où les tableaux vivent en VRAM et les opérations sont exécutées par des kernels CUDA plutôt que sur le CPU.

C'est une architecture typique de « TP démo GPU dans Colab avec frontend séparé » — un pattern qui revient dans plusieurs TP de la même série.

2. Génération du dataset

Plutôt que de fournir un corpus réel, le notebook **génère synthétiquement** 10 000 phrases en combinant de façon combinatoire quatre listes de vocabulaire thématique :

- 10 sujets (IA, deep learning, calcul parallèle, informatique quantique...)
- 10 verbes d'action (« permet de résoudre », « accélère le traitement de »...)
- 10 objets (« calculs matriciels complexes », « séquences d'ADN volumineuses »...)
- 10 contextes (« en utilisant des cartes graphiques GPU », « grâce à des architectures CUDA »...)

Avec $10 \times 10 \times 10 \times 10 = 10\,000$ combinaisons possibles, le générateur produit exactement 10 000 phrases uniques par simple parcours imbriqué (for s in sujets: for v in verbes: ...), avec une boucle de secours aléatoire si jamais moins de 10 000 combinaisons étaient disponibles (ici superflue puisque le compte tombe juste). Chaque document est stocké avec un id, un titre généré (Document Scientifique #0001) et le text, puis sauvegardé dans dataset.json.

Remarque méthodologique : ce corpus synthétique, bien que pratique pour avoir un volume de données contrôlé, a un vocabulaire très restreint et répétitif (seulement 40 expressions de base). Cela signifie que la matrice document-terme sera très creuse et que beaucoup de documents auront des scores de similarité identiques ou très proches — utile pour tester la mécanique du système, mais peu représentatif d'un cas réel de recherche documentaire.

3. Moteur de recherche vectorielle CPU/GPU

C'est le cœur technique du notebook, et la partie la plus aboutie.

a) Détection robuste du GPU

python

try:

```
import cupy as cp

test_gpu = cp.asarray([1.0, 2.0, 3.0])

GPU_AVAILABLE = True
```

except Exception as e:

...

Le notebook ne se contente pas de vérifier que cupy s'importe : il force une **allocation réelle en VRAM** (cp.asarray) pour déclencher immédiatement une erreur explicite si le runtime Colab n'a pas de GPU attaché, plutôt que de planter plus tard au milieu d'un calcul. C'est une bonne pratique de *fail-fast* accompagnée d'un message pédagogique détaillé expliquant comment activer le GPU dans Colab.

b) Vectorisation Bag-of-Words

- Nettoyage du texte (minuscule + suppression de la ponctuation via regex).
- Construction du vocabulaire global (set de tous les mots uniques).
- Construction d'une matrice document-terme dense doc_matrix_cpu de forme (10000, vocab_size) en float32, remplie par comptage de fréquence.
- Normalisation L2 de chaque ligne (avec protection contre la division par zéro via norms_cpu[norms_cpu == 0] = 1e-9), ce qui permet ensuite de calculer la similarité cosinus par un simple produit scalaire.
- Si le GPU est disponible, la matrice normalisée est transférée en VRAM (cp.asarray(doc_matrix_cpu_norm)) **une seule fois**, en dehors de la boucle de recherche — bon réflexe de performance, puisque le transfert CPU→GPU est coûteux et ne doit pas être répété à chaque requête.

c) **search_cpu()** **et** **search_gpu()**

Les deux fonctions suivent exactement la même logique (vectoriser la requête, calculer les scores par produit matriciel, trier, retourner le top-k), ce qui garantit une comparaison équitable :

- `search_cpu` : `np.dot(matrix_subset, query_vec_norm)` puis `np.argsort`.
- `search_gpu` : équivalent avec `cp.dot` et `cp.argsort`, plus le transfert du vecteur requête vers le GPU (`cp.asarray`) et le rapatriement des résultats vers le CPU (`cp.asnumpy`) à la fin.

Point notable : `search_gpu()` contient son propre `try/except` qui rebascule vers `search_cpu()` en cas d'erreur pendant le calcul (pas seulement à l'initialisation) — un **double niveau de tolérance aux pannes**, ce qui est plutôt rigoureux pour un TP.

Un paramètre `limit_docs` permet de restreindre la taille du corpus utilisé pour la recherche, ce qui laisse la possibilité de faire un vrai benchmark à tailles variables (1000, 5000, 10000 documents...), même si le notebook ne l'exploite pas explicitement dans une boucle de mesure comme le TP précédent sur CPU vs GPU.

4. API Flask

Trois routes REST, cohérentes avec les besoins d'un frontend de démonstration :

Route	Méthode	Rôle
<code>/status</code>	GET	Renvoie l'état du backend et si le GPU est actif
<code>/search</code>	POST	Effectue une recherche, avec choix explicite du mode (gpu ou cpu) dans le corps de la requête
<code>/benchmark</code>	POST	Compare les temps CPU et GPU sur une requête fixe, avec paramètre <code>limit_docs</code>

La route `/search` gère correctement le cas d'une requête vide (400 Bad Request) et retourne le mode réellement utilisé (utile si le client demande gpu mais que le GPU n'est pas dispo — la réponse indique honnêtement `"mode": "cpu"`).

5. Exposition réseau via ngrok

Le tunnel ngrok est configuré via un token à renseigner manuellement (`NGROK_TOKEN = "COLLE_TON_TOKEN_ICI"`), avec une vérification basique (`len(NGROK_TOKEN) < 20`) empêchant un lancement silencieux avec un token invalide. Le serveur Flask est lancé dans un **thread séparé** (`threading.Thread(target=run_flask).start()`) pour ne pas bloquer l'exécution du notebook — c'est la bonne façon de faire tourner un serveur long-running dans Colab.

Synthèse — Points forts

- Architecture claire et bien séparée : données → moteur de calcul → API → réseau.
 - Gestion des erreurs GPU à deux niveaux (initialisation + exécution), avec messages pédagogiques.
 - Cohérence stricte entre les implémentations CPU et GPU pour une comparaison valide.
 - Bonnes pratiques réseau (CORS, thread séparé, vérification du token).
1. **Corpus synthétique très répétitif** (40 briques de vocabulaire combinées) : peu représentatif d'un cas d'usage réel, ce qui limite la portée des résultats de pertinence de recherche (même si l'objectif du TP est la performance, pas la qualité sémantique).
 2. Le paramètre `limit_docs` permettrait une vraie étude de scalabilité (comme dans le TP CPU vs GPU précédent) mais n'est pas exploité dans une boucle de mesure automatisée — ce serait un ajout naturel pour enrichir ce TP.
 3. Token ngrok en clair dans le notebook : à retirer avant tout partage public du fichier.